We need a proper **Link Manager** in the blog generator, so AI does not guess links randomly.

Goal

When a blog is generated, the system should:

1. load approved internal and external links
2. match the best links to the blog topic
3. give those links to the AI in structured form
4. force the AI to place them naturally in the article
5. save which links were actually used

1. Build a link database

Create a table or JSON source with approved links.

Each internal link should have:

- url
- page title
- category
- target keywords
- allowed anchor text variations
- status (active/inactive)

Example:


```
{

  "internal_links": [

    {

      "id": "vara-license",

      "url": "https://aston.ae/business-setup-dubai/vara-license/",

      "title": "VARA license",

      "category": "crypto",

      "keywords": ["vara", "crypto", "virtual assets", "dubai crypto licence"],

      "anchors": ["VARA licence", "crypto licence in Dubai", "VARA licensing"]

    },

    {
```

"id": "business-setup-dubai",
"url": "https://aston.ae/business-setup-dubai/",
"title": "Business setup Dubai",
"category": "company formation",
"keywords": ["business setup", "company formation", "start business in dubai"],
"anchors": ["business setup in Dubai", "company formation in Dubai"]
}
],
"external_links": [
{
"id": "dfsa",
"url": "https://www.dfsa.ae",
"title": "DFSA",
"category": "regulator",
"keywords": ["dfsa", "financial regulation", "difc regulation"],
"anchors": ["Dubai Financial Services Authority", "DFSA"]
}
]
}

2. Add Link Manager logic before content generation

Before sending the prompt to OpenAI:

Input:

- topic
- focus keyword
- optional source material
- full link database

Logic:

- score internal links against the topic/keyword
- select top 3–5 internal links
- select top 1–2 external links if relevant
- avoid irrelevant links
- avoid duplicate categories unless needed

Simple scoring is enough at first:

- keyword match in topic
- keyword match in outline
- category match

3. Pass selected links into the prompt

Do not send the entire sitemap every time.
Only send the selected links.

Example prompt input block:


```
{
  "selected_internal_links": [
    {
      "url": "https://aston.ae/business-setup-dubai/",
      "anchors": ["business setup in Dubai", "company formation in Dubai"]
    },
    {
      "url": "https://aston.ae/business-setup-dubai/vara-license/",
      "anchors": ["VARA licence", "crypto licence in Dubai"]
    }
  ],
  "selected_external_links": [
    {
      "url": "https://www.dfsa.ae",
      "anchors": ["Dubai Financial Services Authority", "DFSA"]
    }
```

```
  ]
}
```

4. Update the prompt rules

The prompt should explicitly say:

- Use 3–5 internal links from the provided list
- Use 1–2 external links only if relevant
- Use anchor text from the provided anchor options
- Place links naturally inside sentences
- Spread them across the article
- Do not cluster all links in one paragraph
- Do not invent URLs
- Do not use raw URLs as anchor text

5. Force structured output for used links

The AI should return both:

1. the article HTML
2. a link usage report

Example output:

```
{
  "internal_links_used": [
    {
      "anchor": "business setup in Dubai",
      "url": "https://aston.ae/business-setup-dubai/"
    },
    {
      "anchor": "VARA licence",
      "url": "https://aston.ae/business-setup-dubai/vara-license/"
    }
  ],
  "external_links_used": [
```

```
    {
      "anchor": "Dubai Financial Services Authority",
      "url": "https://www.dfsa.ae"
    }
  ]
}
```


This is important so we can:

- verify what was inserted
- store it in DB
- show it in admin UI
- debug bad outputs

6. Add admin UI for Link Manager

We need a simple panel where we can:

- add/edit/remove links
- add keyword tags
- add anchor text variations
- set category
- disable bad/outdated links

Suggested columns:

- Title
- URL
- Type (internal/external)
- Category
- Keywords
- Anchors
- Status

7. Save link usage per blog post

When a post is generated, save:

- post id
- topic
- selected links
- actual links used
- anchors used

This will help later for:

- SEO audits
- internal link analysis
- avoiding overuse of the same anchor

8. Basic first version

Version 1 can be simple:

Backend steps:

1. user enters topic
2. backend matches top internal/external links from DB
3. backend sends selected links into prompt
4. AI writes article + returns usage report
5. save article + links used
6. push article to WordPress

That is enough to make linking much better immediately.

9. Better version later

Later we can improve it with:

- semantic matching using embeddings
- link suggestions per section, not only per article
- automatic checking that link targets are live
- anchor diversity rules
- topic cluster logic

10. Important rule

AI should never freely decide links from memory.
It should only use links supplied by our Link Manager.

That is the whole point:
controlled links in, controlled links out.

If needed, I can also turn this into:

1. a DB schema
2. API request/response format
3. exact prompt format for OpenAI

Goal

Before generating the full blog, the system must first create a **structured blog blueprint**.

This ensures:

- consistent layout (like our existing Aston blogs)
- correct length (2,100–2,300 words)
- proper section distribution
- better Elementor rendering
- better SEO structure

---

1. Core concept

Do NOT generate the full blog immediately.

Instead, use a **two-step generation process**:

1. Generate structured outline (blueprint)
2. Generate full blog based on that blueprint

---

2. Input to structure engine

The structure engine receives:

- topic
- focus keyword
- secondary keywords
- target audience
- optional source material
- selected internal links (from Link Manager)
- selected external links

---

3. Output (structure blueprint)

The AI should return a structured JSON object like this:

```
{
  "title": "",
  "focus_keyword": "",
  "meta_title": "",
```

```
  "meta_description": "",
  "slug": "",
  "estimated_word_count": 2200,
  "sections": [
    {
      "type": "intro",
      "heading": "",
      "target_words": 150
    },
    {
      "type": "key_takeaways",
      "heading": "Key takeaways",
      "items_count": 5
    },
    {
      "type": "section",
      "heading": "",
      "target_words": 300,
      "subsections": [
        {
          "heading": "",
          "target_words": 120
        }
      ]
    },
    {
      "type": "quote",
      "target_words": 60
    },
    {
      "type": "section",
      "heading": "",
      "target_words": 300
    },
    {
      "type": "faq",
      "heading": "Frequently asked questions",
      "items_count": 4
    },
    {
      "type": "cta",
      "heading": "Aston VIP’s role in your licensing journey",
      "target_words": 180
    }
  ]
}
```


---

4. Required structure rules

The structure engine MUST enforce:

- total target: 2,100–2,300 words
- intro: 150–200 words
- minimum 5 H2 sections
- minimum 2 H3 subsections
- 1 key takeaways block (5 points)
- 1 quote block
- 1 FAQ section (3–5 questions)
- 1 CTA section (Aston VIP section)
- 3–4 image placements (can be added as section type: "image")

---

5. Section types to support

Each section must have a "type":

- intro
- key_takeaways
- section (H2)
- subsection (H3)
- quote
- faq
- cta
- image

This allows frontend mapping later.

---

6. How it connects to writing step

After structure is generated:

The full blog generator should receive:

- full structure JSON
- selected links (from Link Manager)
- original prompt rules (master prompt)

Then AI writes the full article by filling each section.

---

7. Writing rules per section

The writing step must respect:

- target word count per section
- heading hierarchy (H1 → H2 → H3)
- link placement across early sections
- natural paragraph spacing (3–4 lines)
- Aston tone and SEO rules

---

8. Why this is required

Without this step:

- AI writes inconsistent layouts
- sections become too short or too long
- Elementor output looks flat or broken
- SEO structure is weaker

With this step:

- every blog follows same high-quality format
- easier mapping to Elementor components
- consistent reading experience
- better SEO performance

---

9. Basic implementation flow

1. user enters blog inputs
2. backend calls structure engine (AI)
3. receives structured blueprint
4. backend calls content generator with blueprint
5. receives full article + metadata
6. send to WordPress

---

10. Version 1 scope

Keep it simple:

- static section rules
- basic word targets

- no need for complex AI logic yet

We can improve later with:

- dynamic structures per topic
- section scoring
- performance-based adjustments

---

11. Important rule

The content generator MUST follow the structure.

AI should not:

- skip sections
- merge sections
- ignore word targets

Structure is the source of truth.

Correction for **Step 3: Image Engine**

Goal

Images must be generated from the **actual blog topic and section meaning**, not from a generic business style prompt.

That means the image system should understand:

- what the full article is about
- what each section is about
- what visual would best support that exact section

The brand style still matters, but **topic relevance comes first**.

---

1. How image generation should work

The image engine should use:

- blog title
- focus keyword
- section heading
- section summary / section text

- article category
- brand style rules

So the flow should be:

1. generate blog structure
2. generate article content
3. extract image opportunities from the actual content
4. generate image prompts based on those sections
5. create and place images

Do NOT create image prompts before the article structure/content is known.

---

2. Image prompts must be section-based

Each image should be tied to a specific section.

Example structure:


```
{
  "image_blocks": [
    {
      "placement": "hero",
      "related_heading": "How to set up a BVI company",
      "section_summary": "Overview of BVI company formation for international founders",
      "image_prompt": "Realistic visual of international business incorporation concept related to BVI company formation, professional corporate setting, relevant to offshore company setup, high detail, realistic photography style",
      "alt_text": "BVI company formation for international business owners"
    },
    {
      "placement": "after_section_2",
      "related_heading": "Bank account opening requirements",
      "section_summary": "Explaining compliance checks and bank onboarding",
      "image_prompt": "Realistic professional banking compliance meeting, documents under review, corporate onboarding environment, relevant to international company bank account opening, realistic photography",
      "alt_text": "Bank account opening process for an international company"
    }
  ]
}
```


---

3. Image prompt source must come from the blog itself

The system should derive image ideas from:

- H1
- H2
- H3
- key facts in the section
- practical concepts mentioned in the article

Examples:

If blog is about:

- **BVI company formation** → show incorporation, compliance, international business ownership
- **US E2 visa** → show investor documents, business plan review, visa-related legal consultation
- **VARA licence** → show regulatory review, digital asset compliance, licensing preparation
- **Cyprus company with bank account** → show company registration workflow, banking review, corporate structuring

So the prompt should reflect the subject, not just “business people in Dubai office”.

---

4. Brand style is a secondary layer

After topic relevance is defined, then apply Aston style rules:

- realistic photography
- premium corporate feel
- clean and trustworthy
- professional settings
- no fake AI look
- no text in image
- no surreal effects

So the logic is:

topic relevance first
+
brand style second

---

5. What the system should avoid

Do NOT generate:

- generic handshake photos
- random skyline photos
- unrelated office scenes
- repeated “people in meeting room” images for every article

That makes the blogs feel fake and low quality.

Each image must visually support the actual article section.

---

6. Better implementation logic

For every blog, create:

Hero image

Based on:

- full article title
- focus keyword
- main intent of article

Section images

Based on:

- exact section heading
- section summary
- most visual concept in that section

So image planning should happen after content structure is defined.

---

7. Suggested backend flow

1. generate structure
2. generate article
3. extract image-worthy sections
4. create one image brief per chosen section
5. send brief to image API
6. upload to WordPress
7. place image in mapped section

---

8. Important rule

The image engine should not use one fixed prompt template for all blogs.

It should use a template with variables from the actual article, for example:

Create a realistic editorial-style image for a blog section about: [section_heading].
Context: [section_summary].
Topic: [focus_keyword].
Style: realistic, premium, professional, trustworthy, no text, no surreal effects.

---

9. Result we want

We want images that feel like they belong to the article itself.

Not:

- generic corporate filler

But:

- visually relevant editorial images that support the exact topic being discussed

Now we need to build **Step 4: SEO + metadata engine**

Goal

After the blog content and images are generated, the system must create and map all SEO fields correctly for WordPress and Yoast.

This step is required so the post is not just well written, but also:

- properly structured for search engines
- correctly filled in Yoast fields
- consistent in slug, title, meta description, and excerpt
- ready for publishing without manual cleanup

---

1. Core concept

The system should generate SEO data as a separate structured layer.

Do NOT rely on the article body alone.

We need explicit output for:

- post title
- SEO title
- slug
- meta description
- focus keyword
- secondary keywords
- excerpt
- image alt text
- FAQ schema data if used

---

2. Input to SEO engine

The SEO engine receives:

- blog title
- full article content
- focus keyword
- secondary keywords
- internal links used
- external links used
- FAQ content if present

---

3. Required output

The system should return something like:


```
{
  "post_title": "How to set up a BVI company",
  "seo_title": "How to set up a BVI company in 2026 | Aston",
  "slug": "how-to-set-up-a-bvi-company",
  "meta_description": "Learn how to set up a BVI company, the main compliance points,
costs, and how Aston supports international founders.",
  "focus_keyword": "BVI company formation",
  "secondary_keywords": [
    "offshore company setup",
    "BVI company registration",
    "international business company"
  ],
  "excerpt": "A practical guide to BVI company formation, including setup steps, banking,
compliance, and structuring points.",
  "faq_schema": [
    {
      "question": "How long does BVI company formation take?",
```

```
    "answer": "..."
  }
 ]
}
```

---

4. Rules for each SEO field

Post title

- natural headline for readers
- should match H1 or be very close

SEO title

- may differ slightly from H1
- should include focus keyword
- should be written for search results
- keep within sensible title length

Slug

- lowercase
- hyphenated
- clean
- no stop-word clutter
- based on focus keyword and title

Meta description

- clear summary of the article
- include focus keyword naturally
- written for click-through rate
- no keyword stuffing

Focus keyword

- one main keyword only

Secondary keywords

- related phrases already used naturally in article

Excerpt

- short clean summary for WordPress archive pages

---

5. Yoast mapping

The backend must map generated fields into WordPress / Yoast correctly.

Typical mapping should include:

- post_title → WordPress title
- slug → post_name
- excerpt → post_excerpt
- `_yoast_wpseo_title` → seo_title
- `_yoast_wpseo_metadesc` → meta_description
- `_yoast_wpseo_focuskw` → focus_keyword

If FAQ is included, also generate schema-ready data if your WP setup supports it.

---

6. Validation before publishing

Before sending to WordPress, run checks:

- focus keyword appears in title
- focus keyword appears in intro
- focus keyword appears in at least one H2
- meta description exists
- slug exists
- internal links exist
- at least one external authority link exists if relevant
- image alt text exists
- FAQ exists when article type requires it

This can be simple rule-based validation first.

---

7. Why this step matters

Without this step:

- Yoast fields may stay empty
- title and metadata may be inconsistent
- slugs may be messy
- articles may look good but perform weakly in search

With this step:

- every article gets complete SEO packaging
- WordPress receives structured metadata
- less manual editing before publishing
- much better consistency at scale

---

8. Basic implementation flow

1. content generator finishes article
2. SEO engine creates metadata fields
3. backend validates required SEO conditions
4. backend maps fields to WordPress + Yoast
5. save or publish post

---

9. Version 1 scope

Keep it simple:

- generate all required SEO fields
- map them into WordPress/Yoast
- validate basic SEO rules only

Later we can improve with:

- title scoring
- meta description scoring
- FAQ schema expansion
- Open Graph fields
- Twitter/X card metadata
- canonical URL handling

---

10. Important rule

SEO metadata must be generated as structured output and stored separately.

Do NOT try to extract it manually later from the article body.

The SEO layer should be its own controlled step.

Now we need to build **Step 5: WordPress + Elementor mapping engine**

Goal

After the blog, links, SEO fields, and images are generated, the system must place everything into WordPress and Elementor in the correct structure.

This step is required so the final post does not look like raw AI text pasted into a page.

It should:

- map content blocks into the correct layout
- preserve Aston blog styling
- place images in the right sections
- insert CTA, FAQ, quote box, and key takeaways correctly
- send SEO fields into the right WordPress / Yoast fields
- create a publish-ready post

---

1. Core concept

Do NOT send one long HTML blob into WordPress and hope Elementor makes it look right.

Instead, the system should map each generated content block into the correct page component.

So the flow becomes:

1. generate structured blog
2. generate SEO fields
3. generate images
4. map each block into Elementor / WordPress fields
5. create the final post

---

2. Input to mapping engine

The mapping engine receives:

- post title
- slug
- full structured content
- SEO metadata
- selected links
- generated images
- alt text

- CTA block
- FAQ block
- quote block
- key takeaways
- final Aston VIP section

---

3. Content must be block-based

The generator should already return block types such as:

- hero
- intro
- key_takeaways
- section
- subsection
- quote
- image
- faq
- cta

The mapping engine should use these block types to decide where content goes.

---

4. Example block structure

```
{
  "content_blocks": [
    {
      "type": "intro",
      "content": "..."
    },
    {
      "type": "key_takeaways",
      "items": ["...", "...", "..."]
    },
    {
      "type": "section",
      "heading": "Why founders use BVI companies",
      "content": "..."
    },
    {
      "type": "image",
      "image_id": 123,
      "alt_text": "BVI company formation for international founders"
    },
```

```
    {
      "type": "quote",
      "content": "..."
    },
    {
      "type": "faq",
      "items": [
        {
          "question": "...",
          "answer": "..."
        }
      ]
    },
    {
      "type": "cta",
      "heading": "Aston VIP’s role in your licensing journey",
      "content": "..."
    }
  ]
}
```

---

5. Elementor mapping logic

The backend should map each block type to a predefined Elementor component or section.

Example:

- hero → title + hero image section
- intro → text editor block
- key_takeaways → styled box / icon list
- section → heading + text block
- subsection → smaller heading + text block
- image → image widget
- quote → quote widget / highlighted callout
- faq → accordion widget
- cta → custom CTA section template

The frontend should not guess layout from raw text.
The layout should come from predefined mappings.

---

6. Best implementation approach

Best option:
Use a predefined Elementor blog template and inject generated values into placeholders.

For example:

- `{{title}}`
- `{{hero_image}}`
- `{{intro}}`
- `{{key_takeaways}}`
- `{{section_1_heading}}`
- `{{section_1_body}}`
- `{{faq_items}}`
- `{{cta_heading}}`
- `{{cta_body}}`

This is much safer than building page layout dynamically from scratch each time.

---

7. WordPress fields to map

At minimum, map:

WordPress core

- post_title
- post_name / slug
- post_excerpt
- featured_image
- post_content or Elementor content JSON

Yoast

- `_yoast_wpseo_title`
- `_yoast_wpseo_metadesc`
- `_yoast_wpseo_focuskw`

Images

- media upload
- alt text
- filename
- featured image assignment

---

8. Styling consistency

The mapping layer must preserve Aston style:

- spacing between sections
- paragraph width
- quote styling
- CTA design
- FAQ appearance
- image placement rhythm
- heading hierarchy

This means design should come from template/components, not from AI formatting.

---

9. Why this step matters

Without this step:

- blogs look flat or broken
- paragraphs become uneven
- sections do not match old Aston posts
- CTA and FAQ look inconsistent
- images appear in awkward positions

With this step:

- every blog follows the same premium layout
- output is consistent even when content changes
- publishing becomes much faster
- AI content feels like Aston content

---

10. Basic implementation flow

1. backend receives generated blog package
2. upload all images to WordPress
3. create or fill Elementor template placeholders
4. map structured blocks into correct sections
5. map SEO fields to Yoast
6. save as draft or publish

---

11. Version 1 scope

Keep it simple:

- use one blog template only
- support fixed block mapping
- use placeholders for main sections
- no dynamic template switching yet

Later we can improve with:

- multiple templates by blog type
- optional blocks
- layout variants
- conditional CTAs
- topic-specific modules

---

12. Important rule

Elementor should be the presentation layer only.

AI should generate structured content.
The backend should decide how that content is placed into the template.

Do NOT rely on AI to generate final visual layout.

Now we need to build **Step 6: Final QA + publishing control**

Goal

Before any blog is pushed live, the system must run a final quality check.

This step is required so we do not publish posts that are:

- too short
- badly formatted
- missing links
- missing SEO fields
- using weak images
- inconsistent with Aston style
- factually weak or obviously AI-looking

The purpose is simple:
catch problems before publishing

---

1. Core concept

The system should not publish immediately after generation.

Instead, after content, SEO, links, images, and Elementor mapping are complete, run a final QA step.

This QA step should validate that the output matches all required rules.

Only after passing QA should the post:

- be saved as draft
- or be published automatically, depending on settings

---

2. What QA should check

The QA engine should validate:

Content quality

- total word count is within target range
- intro exists
- minimum number of H2 sections exists
- minimum number of H3 subsections exists
- FAQ exists if required
- CTA exists
- Aston VIP section exists
- paragraphs are not too long
- article does not feel obviously unfinished

SEO

- focus keyword exists
- focus keyword appears in title
- focus keyword appears early in intro
- focus keyword appears in at least one H2
- meta title exists
- meta description exists
- slug exists
- excerpt exists

Links

- 3–5 internal links exist
- internal links are from approved list
- 1–2 external links exist if required
- no broken raw URLs in body
- anchor text is descriptive

Images

- featured image exists
- section images exist
- alt text exists
- filenames are descriptive
- image placement count is correct

Layout / structure

- key takeaways block exists
- quote block exists
- FAQ block renders properly
- CTA block renders properly
- no missing placeholders in Elementor template
- no empty sections

---

3. Suggested QA output

The QA step should return something like:

```
{
  "status": "pass",
  "score": 92,
  "checks": {
    "word_count": true,
    "h2_count": true,
    "h3_count": true,
    "focus_keyword_in_title": true,
    "focus_keyword_in_intro": true,
    "internal_links_count": true,
    "external_links_count": true,
    "meta_description_exists": true,
    "faq_exists": true,
    "cta_exists": true,
    "images_exist": true,
    "elementor_placeholders_filled": true
  },
  "warnings": [
    "One subsection is slightly short",
    "Meta description could be improved for CTR"
  ],
  "blocking_issues": []
}
```

---

4. Publish rules

We should have publish logic like this:

Pass

- save as draft or publish automatically

Pass with warnings

- save as draft and flag for review
- or publish only if auto-publish mode allows warnings

Fail

- do not publish
- regenerate or send to review queue

---

5. Important checks to make blocking

These should stop publishing immediately:

- word count far below minimum
- missing title
- missing slug
- missing meta description
- missing focus keyword
- missing internal links
- broken Elementor placeholders
- missing CTA section
- no featured image
- invalid JSON / invalid block structure

---

6. Optional style checks

Later we can also add checks for:

- banned phrases from master prompt
- repeated phrasing
- robotic sentence patterns
- overuse of same keyword
- weak intros
- generic conclusions

This can start simple with rule-based detection.

---

7. Why this step matters

Without this step:

- bad posts eventually go live
- your team still has to manually inspect everything
- scaling becomes risky
- small failures break trust and design consistency

With this step:

- quality becomes predictable
- weak posts are caught before publish
- automation becomes safer
- review effort becomes much lower

---

8. Basic implementation flow

1. article package is generated
2. mapping to WordPress/Elementor is completed
3. QA engine runs checks
4. result returned as pass / warning / fail
5. backend decides:
    - publish
    - save draft
    - regenerate
    - send for manual review

---

9. Version 1 scope

Keep it simple:

- rule-based validation
- pass / warning / fail
- basic score
- manual review if failed

Later we can improve with:

- LLM-based editorial review
- fact checking against source material
- plagiarism risk review
- brand tone scoring
- conversion quality scoring

---

10. Important rule

QA must happen **after** the full post is assembled in its final state.

That means it should check:

- final text
- final links
- final images
- final metadata
- final Elementor output

Not just the raw AI text.

Now we need to build **Step 7: Source + research engine**

Goal

The system must handle different kinds of blog creation properly.

Not every blog starts the same way. Sometimes we write from:

- a topic only
- source material
- an old Aston blog
- a competitor article used for inspiration
- internal notes or pasted research

This step makes sure the system knows how to use those inputs correctly.

---

1. Core concept

The system should support different generation modes.

At minimum, add these modes:

Mode A — Topic only

Use when we only provide:

- topic
- focus keyword
- audience
- optional secondary keywords

The system researches and builds the article from scratch.

Mode B — Source-assisted original article

Use when we provide:

- topic
- source article / pasted text / file / URL
- focus keyword
- audience

The system should use the source as reference material only, then write a fully original Aston article.

Mode C — Improve existing Aston article

Use when we provide:

- old Aston article
- updated keyword
- updated links
- improved structure requirements

The system should refresh and improve the content without losing Aston style.

Mode D — Notes to article

Use when we provide:

- rough notes
- bullet points
- client notes
- developer notes

The system should turn them into a full article.

---

2. Why this step matters

Right now, if all tasks use one giant prompt, the model may treat very different inputs the same way.

That causes problems such as:

- competitor article being paraphrased too closely
- source material being copied structurally
- old Aston articles being rewritten badly
- notes not being expanded correctly
- weak factual grounding

We need a dedicated source engine so the system knows what each input means.

---

3. Input fields needed

The source engine should accept:

- generation_mode
- topic
- focus_keyword
- target_audience
- secondary_keywords
- source_text
- source_urls
- source_files
- existing_aston_article
- notes
- allowed_external_sources
- internal link candidates

---

4. Behaviour rules by mode

Mode A — Topic only

The system should:

- generate article from the topic
- use approved internal links
- use approved external sources only if needed
- create original structure and content

Mode B — Source-assisted original article

The system should:

- extract useful facts, themes, and questions from source material
- NOT paraphrase sentence by sentence

- NOT copy section order too closely
- create a new Aston-specific outline
- write a fully original article

Mode C — Improve existing Aston article

The system should:

- preserve useful original points
- improve structure
- improve SEO
- update outdated sections
- strengthen links, FAQ, CTA, and metadata

Mode D — Notes to article

The system should:

- expand notes into a complete outline
- identify missing logical sections
- create a polished article in Aston tone

---

5. Source processing stage

Before the final article is written, the source engine should first create a structured source brief.

Example:

```
{
  "mode": "source_assisted",
  "source_summary": "Article explains the legal and practical process for obtaining an E2 investor visa through business investment.",
  "key_facts": [
    "...",
    "...",
    "..."
  ],
  "questions_answered": [
    "...",
    "...",
    "..."
  ],
  "useful_angles": [
    "...",
    "...",
```

```
    "..."
  ],
  "avoid_reusing": [
    "distinctive phrasing",
    "original heading structure",
    "specific repeated examples"
  ]
}
```

This brief is then used by the structure engine and writing engine.

---

6. Competitor content rule (VERY IMPORTANT)

If a competitor article or external blog is provided, the system must treat it as:

reference material only

It must not:

- rewrite line by line
- preserve the same article flow
- reuse distinctive wording
- create near-duplicate content

Instead it should:

- extract insights
- identify useful subtopics
- build a new Aston outline
- write an original article

This is one of the most important rules in the whole system.

---

7. Research handling

For topic-only generation, the source engine should be able to gather research inputs before writing.

That can include:

- approved regulator websites
- official government sources
- approved reference pages
- internal Aston pages for context

This research should then be summarised into a structured brief before writing begins.

---

8. Output of source engine

The source engine should return something like:

```
{
  "generation_mode": "topic_only",
  "research_brief": {
    "main_topic": "",
    "search_intent": "",
    "key_points_to_cover": [],
    "audience_needs": [],
    "useful_facts": [],
    "approved_external_sources": []
  }
}
```

or for source-assisted mode:

```
{
  "generation_mode": "source_assisted",
  "source_brief": {
    "main_topic": "",
    "source_summary": "",
    "key_facts": [],
    "content_opportunities": [],
    "questions_to_answer": [],
    "do_not_copy_rules": []
  }
}
```

---

9. Basic implementation flow

1. user selects generation mode
2. backend gathers all source inputs
3. source engine creates structured brief
4. structured brief is sent into:
    - link manager
    - structure engine
    - content generator
    - SEO engine

So the source engine becomes the foundation layer for the rest of the pipeline.

---

10. Version 1 scope

Keep it simple:

- support 4 generation modes
- process pasted text, URLs, and notes
- create structured source brief
- pass brief into later stages

Later we can improve with:

- multiple source comparison
- source confidence scoring
- fact extraction and validation
- update detection for old Aston blogs
- topic gap analysis

---

11. Important rule

The system must always know what kind of source it is handling before writing begins.

Do NOT treat:

- a topic
- a source article
- old Aston content
- rough notes

as if they are the same input type.

That is what causes messy outputs.

Now we need to build **Step 8: Prompt builder + orchestration layer**

Goal

We now have several engines:

- Link Manager
- Content Structure Engine
- Image Engine
- SEO + metadata engine
- WordPress + Elementor mapping
- Final QA

- Source + research engine

This step connects everything into one controlled workflow.

The purpose is:

- assemble the right prompt for each task
- run each step in the correct order
- pass outputs cleanly between modules
- avoid one giant messy prompt
- make the whole system scalable and easier to maintain

---

1. Core concept

Do NOT use one huge manual prompt for every task.

Instead, the backend should act as an orchestration layer that builds prompts dynamically from structured inputs.

So the flow becomes:

1. collect user inputs
2. detect generation mode
3. run source engine
4. run link manager
5. run structure engine
6. run writing engine
7. run image engine
8. run SEO engine
9. run WordPress/Elementor mapping
10. run QA
11. save draft or publish

This orchestration layer is what turns separate components into one blog engine.

---

2. What the prompt builder should do

The prompt builder should assemble the final prompts from smaller modules.

For example:

System rules

- Aston tone

- UK English
- banned phrases
- SEO expectations

Source rules

- topic-only
- source-assisted
- Aston rewrite
- notes-to-article

Link rules

- selected internal links
- selected external links
- anchor text options

Structure rules

- required sections
- target word counts
- image blocks
- CTA / FAQ / quote requirements

Output rules

- HTML
- JSON metadata
- link usage report
- image prompt data
- SEO field data

This should all be assembled automatically by backend logic.

---

3. Why this is required

If all instructions live in one giant prompt field:

- difficult to maintain
- hard to debug
- hard to improve one part without breaking another
- impossible to scale cleanly
- different blog modes get mixed together

With orchestration:

- each module has one responsibility
- prompts stay cleaner
- outputs stay more predictable
- easier for dev to improve the system over time

---

4. Suggested architecture

The orchestrator should manage these stages:

Stage A — Intake

Collect:

- topic
- keyword
- audience
- source input
- generation mode
- optional notes

Stage B — Source brief

Run source engine and create structured source brief.

Stage C — Link selection

Run Link Manager to choose internal/external links.

Stage D — Structure blueprint

Generate structured article outline with target sections and word counts.

Stage E — Article generation

Generate full article based on:

- source brief
- selected links
- structure blueprint
- Aston writing rules

Stage F — Image generation

Generate image briefs and image assets from article sections.

Stage G — SEO metadata

Generate title, slug, meta description, focus keyword, excerpt, schema data.

Stage H — Mapping

Push content into WordPress + Elementor template.

Stage I — QA

Run final validation.
Then:

- publish
- save as draft
- or send for review

---

5. Prompt builder inputs

The prompt builder should receive structured data like:

```
{
  "generation_mode": "source_assisted",
  "source_brief": {},
  "selected_internal_links": [],
  "selected_external_links": [],
  "structure_blueprint": {},
  "brand_rules": {},
  "seo_rules": {},
  "output_schema": {}
}
```

Then it builds the exact prompt needed for the current stage.

---

6. Important implementation rule

Each stage should have its own prompt.

For example:

- source analysis prompt
- structure generation prompt
- writing prompt
- image prompt
- SEO prompt
- QA prompt

Do NOT try to do everything in one call.

That usually reduces quality and makes debugging much harder.

---

7. Error handling

The orchestrator should handle failures gracefully.

Examples:

- if no suitable external links found → continue without them if allowed
- if image API fails → use fallback provider or stock fallback
- if QA fails → keep as draft and flag issue
- if WordPress mapping fails → save generated package for retry

This is important so one failed step does not destroy the whole workflow.

---

8. Data passing between modules

Each module should return structured output that becomes the next module's input.

Example:

- source engine → source_brief
- link manager → selected_links
- structure engine → structure_blueprint
- writer → article_blocks + link_report
- image engine → image_assets + alt_text
- SEO engine → metadata package
- QA engine → pass/warn/fail result

This should all be stored in a consistent internal schema.

---

9. Logging and traceability

The orchestration layer should save:

- original user input
- source mode used
- prompts sent to each stage
- outputs returned from each stage
- selected links

- generated metadata
- QA result
- final WordPress post id

This makes debugging much easier and helps improve prompts later.

---

10. Version 1 scope

Keep it simple:

- sequential pipeline
- one prompt per stage
- static prompt templates
- simple fallback rules
- basic logs

Later we can improve with:

- retries per module
- confidence scoring
- multiple model routing
- cost optimisation
- topic-specific flows
- human approval checkpoints

---

11. Important rule

The orchestration layer should be the source of truth for the full process.

That means:

- prompts are assembled in backend
- modules do one job each
- outputs are structured
- no manual giant all-in-one prompt is needed in production

This is what makes the whole system stable.

Update for **Step 9: Admin UI / control panel**

Goal

The admin UI must not only manage single blog creation.

It must also control:

- automatic daily generation
- topic queue management
- review workflow
- publishing rules
- blog volume per day
- generation status across all posts

So this becomes the main operating dashboard for the full blog engine.

---

1. Core concept

We need one internal control panel where the team can manage both:

Manual workflow

- create one blog
- review one blog
- edit one blog
- publish one blog

Automated workflow

- schedule 3, 4, 5, or 10 blogs per day
- control queue
- review generated outputs
- approve / reject / retry failed posts
- control draft vs publish behaviour

This means Step 9 should become the place where the whole system is operated.

---

2. Main sections of the control panel

The admin UI should include these sections:

A. Dashboard

Show:

- blogs generated today
- blogs queued

- blogs failed
- blogs in draft
- blogs published
- QA warnings
- scheduler status
- next scheduled generation run

B. Blog setup

Fields:

- topic
- focus keyword
- secondary keywords
- target audience
- article type
- generation mode

C. Source material

Fields:

- source URL
- pasted article text
- uploaded file
- old Aston article
- rough notes
- source handling mode

D. Link management

Show:

- selected internal links
- selected external links
- anchor text options
- option to manually replace or remove links

E. Structure preview

Show:

- generated title
- sections
- H2/H3 structure
- word targets
- key takeaways
- FAQ

- CTA
- image placements

F. Image preview

Show:

- planned image prompts
- generated images
- alt text
- option to regenerate one image only

G. SEO preview

Show:

- post title
- SEO title
- slug
- meta description
- excerpt
- focus keyword
- secondary keywords
- FAQ schema

H. QA report

Show:

- pass / warning / fail
- score
- failed checks
- warnings
- publish readiness

I. Publishing

Buttons:

- generate
- regenerate
- save draft
- push to WordPress
- publish
- send for review

J. Scheduler / automation control

This is the new required section.

Show and control:

- blogs per day
- generation times
- active / paused status
- draft only / auto publish / approval mode
- maximum blogs per run
- retry failed blogs setting
- topic source
- queue depth
- last run result
- next run time

K. Topic queue manager

Show:

- all queued topics
- topic priority
- assigned generation mode
- source input attached
- status
- created date
- scheduled date
- option to reorder queue
- option to pause specific topics
- option to force-run one topic

---

3. Scheduler controls needed in UI

The team must be able to configure automation from the UI directly.

Required controls:

- enable / disable scheduler
- set blogs per day
- choose run times
- choose publish mode
- choose topic source
- choose whether failed posts auto-retry
- choose max posts per batch
- set whether QA warnings block publishing
- pause all automation quickly

Example settings:

- blogs per day: 5
- run times: 08:00, 11:00, 14:00, 17:00, 20:00
- publish mode: draft only
- topic source: queue
- retry failed posts: yes
- auto publish only if QA passes: yes

---

4. Topic queue manager requirements

We need a queue page where the team can manage upcoming content.

Each topic should show:

- topic title
- focus keyword
- generation mode
- source type
- priority
- status
- assigned run
- result link if generated

Statuses should include:

- queued
- scheduled
- processing
- generated
- QA warning
- failed
- draft created
- published
- paused

Actions should include:

- edit
- move up/down in queue
- pause
- resume
- duplicate
- regenerate
- delete

---

5. Blog request statuses

The UI should track blog progress clearly.

Suggested statuses:

- new
- queued
- scheduled
- generating
- structure ready
- content ready
- image ready
- SEO ready
- QA warning
- approved
- draft in WordPress
- published
- failed

This makes the workflow easy to understand for the team.

---

6. Why this matters

Without adding automation controls into Step 9:

- scheduling will live somewhere separate
- team cannot easily control volume
- queue management becomes messy
- failures are harder to handle
- daily publishing is harder to supervise

With these controls in Step 9:

- the team has one place to operate everything
- manual and automatic workflows stay connected
- easier to scale daily volume safely
- easier to monitor quality

---

7. Suggested dashboard widgets

The main dashboard should show:

- posts generated today

- posts scheduled for today
- posts failed today
- posts waiting review
- posts published today
- average QA score
- current scheduler mode
- queue size
- next generation run
- recent failed topics

This gives a quick operational overview.

---

8. Review workflow inside UI

For each generated blog, the team should be able to:

- open full preview
- see source mode used
- see links used
- see images used
- see SEO fields
- see QA report
- approve
- reject
- send back for regeneration
- publish
- keep as draft

This is important because automated generation still needs editorial control.

---

9. Version 1 scope

Keep it simple:

- one dashboard
- one queue page
- one blog detail/review page
- scheduler on/off
- blogs per day setting
- draft-only publish mode
- manual retry for failed blogs

Later we can improve with:

- multi-user roles
- approval workflows
- calendar view
- campaign grouping
- performance analytics
- topic planning by cluster
- prompt version history

---

10. Important rule

The admin UI should be the operating layer for both:

- creating content
- controlling automation

It should not just display blog outputs.
It must also let the team manage:

- what gets generated
- when it gets generated
- how many posts run
- what happens after generation

Now we need to build **Step 10: Backend scheduler + bulk generation logic**

Goal

Step 9 controls automation from the UI.
Step 10 is the backend logic that actually runs it.

This step is responsible for:

- reading the scheduler settings from the admin panel
- pulling topics from the queue
- running the blog pipeline automatically
- handling failures properly
- making sure daily generation works reliably at scale

So this is the execution layer behind the controls from Step 9.

---

1. Core concept

The scheduler should run automatically based on settings defined in the admin UI.

For example:

- 3 blogs per day
- 5 blogs per day
- 10 blogs per day

At each scheduled run, the backend should:

1. read active scheduler settings
2. check if automation is enabled
3. pull the next topics from the queue
4. send each topic through the full blog pipeline
5. save results
6. update statuses
7. publish or save draft depending on rules

---

2. Inputs from Step 9

The backend scheduler should read these settings from the admin panel:

- scheduler enabled / disabled
- blogs per day
- generation times
- publish mode
- max posts per run
- retry failed posts
- topic source
- whether QA warnings block publishing
- queue priority rules

Example settings object:

```
{
  "enabled": true,
  "blogs_per_day": 5,
  "generation_times": ["08:00", "11:00", "14:00", "17:00", "20:00"],
  "publish_mode": "draft_only",
  "max_posts_per_run": 1,
  "retry_failed_posts": true,
  "topic_source": "queue",
  "block_publish_on_qa_warning": true
}
```

---

3. Topic queue processing

The scheduler should pull topics from the queue table.

Each queue record should contain:

- id
- topic
- focus keyword
- secondary keywords
- audience
- generation mode
- source input
- priority
- status
- scheduled_at
- retry_count
- last_error

Possible statuses:

- queued
- scheduled
- processing
- generated
- qa_warning
- failed
- draft_created
- published
- paused

---

4. How each run should work

At every scheduled run:

Step A

Load scheduler settings

Step B

Check:

- is scheduler enabled
- is current time a valid run time
- is there capacity left for today

Step C

Pull next valid topics from queue:

- status = queued or retryable failed
- not paused
- ordered by priority, then created date

Step D

For each selected topic:

- mark as processing
- run full orchestration pipeline
- save output
- update result status

Step E

Continue until:

- run limit reached
- daily quota reached
- no more valid topics

---

5. Important rule

Each blog must still run through the full pipeline individually:

- source engine
- link manager
- structure engine
- writing engine
- image engine
- SEO engine
- mapping engine
- QA engine

Do NOT batch multiple blogs into one model call.

That will reduce quality and make debugging harder.

---

6. Concurrency

Version 1 recommendation:

Safe option

Run one blog at a time

This is easier for:

- logging
- retries
- debugging
- API reliability

Later, you can support limited concurrency such as:

- 2 or 3 jobs in parallel

But version 1 should prioritise control over speed.

---

7. Daily quota logic

The scheduler must track how many blogs were already generated that day.

Example:

- daily target = 5
- 3 already generated today
- next run can process 2 more maximum

This avoids over-generation.

Also reset daily counters at the start of a new day.

---

8. Failure handling

If one blog fails, the whole run must not fail.

Per-topic failure rules:

- catch the error
- save error message
- increment retry_count
- mark status as failed
- continue with next topic

Possible fail sources:

- bad source input
- invalid AI output
- image API failure
- WordPress push failure
- QA fail
- mapping fail

---

9. Retry logic

Retry rules should be configurable.

Suggested version 1:

- max retries per topic = 2
- retry only on technical failures
- do not auto-retry low-quality QA failures unless explicitly enabled

Example:

- image API timeout → retry allowed
- invalid WordPress response → retry allowed
- QA fail because article too weak → keep failed and review manually

---

10. Publish behaviour

The backend must respect publish mode selected in Step 9.

Supported modes:

Draft only

Always create WordPress draft only

Auto publish if pass

Publish only if QA passes with no blocking issues

Manual approval

Create draft and wait for team approval

Recommended version 1:
draft only

That gives the safest rollout.

---

11. Logging and job tracking

Every scheduled job should create logs.

At minimum save:

- run id
- topic id
- start time
- end time
- result status
- QA result
- WordPress post id
- retry count
- error message if failed

This is important for debugging and reporting.

---

12. Scheduler job record

It is useful to have a separate run log table.

Example:

```
{
  "run_id": "run_2026_04_15_0800",
  "started_at": "2026-04-15T08:00:00",
  "completed_at": "2026-04-15T08:22:00",
  "topics_attempted": 3,
  "topics_completed": 2,
  "topics_failed": 1,
  "status": "completed_with_errors"
}
```

This makes monitoring much easier.

---

13. Safeguards

Add these protections:

- do not process paused topics
- do not process topics already marked published
- do not exceed daily quota
- do not run if scheduler disabled
- do not publish if QA blocks it
- do not retry endlessly
- do not run duplicate jobs for same queue item

These rules are important for stability.

---

14. Basic implementation flow

1. cron job / scheduled worker starts
2. reads automation settings from admin panel
3. checks if a run should happen now
4. fetches eligible topics from queue
5. processes each topic through full pipeline
6. saves output + updates statuses
7. creates run log
8. stops when run quota is reached

---

15. Version 1 scope

Keep it simple:

- cron-based scheduler
- queue processing
- one blog at a time
- draft-only publishing
- retry on technical failures only
- basic run logs
- daily quota enforcement

Later we can improve with:

- parallel workers
- smart prioritisation
- multi-site publishing
- cluster-based scheduling
- cost controls
- model routing by content type

---

16. Important rule

Step 10 should only execute the workflow.

The control settings must come from Step 9.

So:

- Step 9 = control center
- Step 10 = execution engine

That separation is important for maintainability.

Now we need to build **Step 11: Topic planning + keyword queue system**

Goal

The scheduler can only work well if it always has good topics ready.

This step is the planning layer that decides:

- what blogs should be written
- which keywords to target
- which topics have priority
- which topics are already covered
- which topics should go into the generation queue next

Without this step, automation becomes random and low quality over time.

---

1. Core concept

We need a structured topic and keyword planning system.

This system should manage:

- master keyword list
- topic ideas
- content clusters
- search intent
- priority
- status
- assignment into the generation queue

So instead of manually inventing topics every day, the system should work from a planned content database.

---

2. What this system should store

Each topic record should include fields like:

- topic id
- main topic
- focus keyword
- secondary keywords
- search intent
- content cluster
- article type
- target audience
- source mode
- priority
- status
- assigned queue status
- linked service page if relevant
- notes
- competitor/source references if any
- publish target date
- published URL once done

Example:


```
{
  "topic_id": "tp_001",
  "topic": "How to set up a BVI company",
  "focus_keyword": "BVI company formation",
  "secondary_keywords": [
    "BVI company registration",
    "offshore company setup",
    "international business company"
  ],
  "search_intent": "informational",
  "content_cluster": "offshore setup",
  "article_type": "guide",
  "target_audience": "international founders",
  "source_mode": "topic_only",
  "priority": "high",
  "status": "planned",
  "queue_status": "not_queued"
}
```


---

3. Why this step matters

Without this system:

- topics get repeated
- keyword targeting gets messy
- posts get generated without strategy
- cluster coverage becomes uneven
- automation runs out of good inputs

With this system:

- topics are planned intentionally
- keywords support each other
- the queue stays full of useful content
- SEO coverage becomes much stronger over time

---

4. Content clusters

The topic planning system should group topics into clusters.

Examples for Aston could be:

- company formation
- offshore structures
- crypto and VARA
- DIFC and ADGM
- visas and residency
- corporate banking
- tax and compliance
- relocation and lifestyle

This helps the team:

- avoid random blog creation
- build authority around core themes
- support internal linking more naturally

Each topic should belong to one cluster.

---

5. Search intent

Each topic should also have search intent.

Possible values:

- informational
- commercial

- transactional
- navigational

Examples:

- “what is a VARA licence” → informational
- “how to get a VARA licence in Dubai” → commercial / transactional
- “Aston DIFC setup services” → navigational / commercial

This is important because article structure and CTA strength may differ by intent.

---

6. Topic lifecycle statuses

Suggested statuses:

- idea
- planned
- approved
- queued
- generating
- draft created
- published
- update needed
- archived

This helps track where each topic is in the system.

---

7. Queue assignment logic

The planning system should be able to push selected topics into the generation queue.

For example:

- choose top 20 approved topics
- sort by priority
- send next 5 into queue
- scheduler consumes from queue

So the planning system feeds Step 10.

This keeps strategy separate from execution.

---

8. Duplicate prevention

The system should check for duplicates before queuing a topic.

Possible checks:

- same focus keyword already published
- very similar title already published
- same topic already queued
- same cluster keyword already covered too recently

This is important to avoid wasted content.

---

9. Updating old content

This system should also support refresh planning.

Not every queued task must be a new article.
Some can be:

- update old Aston blog
- expand thin article
- retarget keyword
- improve internal linking
- add FAQ / schema / images

So topic planner should support:

- new article
- refresh existing article

---

10. Suggested admin features

In the control panel, the team should be able to:

- add topics manually
- import keyword lists
- assign cluster
- assign intent
- set priority
- mark for queue
- filter by status
- filter by cluster
- see published coverage

- see duplicates or conflicts
- move topics into queue

This makes the planning system usable, not just theoretical.

---

11. Basic implementation flow

1. content team adds keyword/topic ideas
2. system stores them in topic planning database
3. topics are reviewed and prioritised
4. approved topics are sent into queue
5. Step 10 scheduler processes queue automatically

So Step 11 becomes the upstream planning layer for the whole engine.

---

12. Version 1 scope

Keep it simple:

- manual topic entry
- keyword fields
- cluster field
- intent field
- priority field
- queue assignment button
- duplicate warning
- status tracking

Later we can improve with:

- keyword import from SEO tools
- automatic clustering
- gap analysis
- topic scoring
- calendar planning
- performance feedback loop

---

13. Important rule

The queue should not be the same thing as the topic planner.

- Topic planner = long-term strategy
- Queue = short-term execution list

That separation is important.

Now we need to build **Step 12: Performance tracking + feedback loop**

Goal

After blogs are published, the system must track how they perform and use that data to improve future content.

This step closes the loop.

Without it:

- you are guessing what works

With it:

- the system learns what ranks, converts, and performs best

---

1. Core concept

We need to track performance for each blog and feed insights back into:

- topic planning (Step 11)
- link strategy (Step 1)
- structure engine (Step 2)
- SEO engine (Step 4)

So the system improves over time instead of staying static.

---

2. What data we should track

For each published blog:

SEO performance

- impressions
- clicks
- average position
- CTR

(Source: Google Search Console)

Traffic

- page views
- sessions
- bounce rate
- time on page

(Source: Google Analytics)

Conversion signals

- form submissions
- calls (if tracked)
- clicks to contact page
- CTA interaction

Content signals

- scroll depth
- engagement per section (if available)
- internal link clicks

---

3. Data storage per blog

Each blog should store:


```
{
  "post_id": 123,
  "url": "...",
  "focus_keyword": "...",
  "cluster": "crypto",
  "published_date": "...",
  "performance": {
    "impressions": 1200,
    "clicks": 85,
    "avg_position": 9.3,
    "ctr": 7.1,
    "time_on_page": 2.4,
```

"bounce_rate": 48
}
}

---

4. Performance classification

We should classify posts automatically:

High performers

- strong impressions
- good rankings
- strong CTR
- good engagement

Medium performers

- some traffic
- average rankings

Low performers

- low impressions
- poor ranking
- low CTR

---

5. What to do with this data

For high performers:

- create related topics in same cluster
- strengthen internal linking to them
- update CTA for conversions
- expand content

For medium performers:

- improve meta title and description
- improve internal linking
- expand sections
- add FAQ
- improve images

For low performers:

- rewrite article
- retarget keyword
- merge with another article
- remove if low value

---

6. Feedback into Topic Planner (Step 11)

The system should use performance data to:

- prioritise clusters that perform well
- reduce focus on weak clusters
- generate similar topic ideas
- identify gaps in content

Example:
If “VARA licence” posts perform well → create more related topics.

---

7. Feedback into Link Manager (Step 1)

- identify which internal links drive clicks
- prioritise linking to high-performing pages
- adjust anchor text strategy

---

8. Feedback into Structure Engine (Step 2)

- identify which structures perform best
- test:
    - more FAQs
    - different CTA placement
    - longer vs shorter sections

---

9. Feedback into SEO Engine (Step 4)

- test different title formats

- test meta description styles
- improve keyword placement rules

---

10. Suggested dashboard (in Step 9 UI)

Add a performance view showing:

- top performing blogs
- lowest performing blogs
- performance by cluster
- performance by keyword
- average position per topic
- CTR comparison
- traffic trends

---

11. Update cycle

The system should periodically review posts:

- 30 days after publish
- 60 days after publish
- 90 days after publish

Then decide:

- keep
- improve
- rewrite
- expand

---

12. Automation ideas (later)

Later we can add:

- automatic rewrite trigger if low performance
- automatic internal linking updates
- automatic title optimisation
- automatic FAQ expansion
- A/B testing of titles

---

13. Version 1 scope

Keep it simple:

- pull data from Google Search Console + Analytics
- store per post
- show basic dashboard
- classify performance
- manual decision on improvements

Later we can automate decisions.

---

14. Important rule

Do NOT rely only on AI assumptions.

Use real performance data to guide:

- what topics to create
- what content to improve
- what strategy to scale